Retail Analytics RFM Segmentation

Customer Segmentation using SQL

Retail Analytics Workflow

Activities

Clean, process, and visualize customer data using Excel , mySQL , Python and Pandas to identify patterns, improve data quality, and generate meaningful business insights.

Business Problem Statement:

A mid-sized Indian retail company wants to analyze its customer base to improve targeted marketing, retain valuable customers, and detect patterns in customer behavior. You have been hired as a data analyst to clean, process, and analyze the customer transaction data to generate actionable insights.

Key goal:

Your goal is to:

- Assess customer demographics and transaction behavior.
- Clean and prepare the data for analysis.
- Conduct RFM (Recency, Frequency, Monetary) analysis.
- Visualize key patterns using charts.

Key Goals:

- Detect incomplete, inconsistent, or duplicate customer records and improve data quality, if applicable.
- Segment customers based on demographics and purchase behavior.
- Identify high-value customers using Recency-Frequency-Monetary (RFM) analysis.
- Visualize customer distribution across geographies, age groups, or income brackets.
- Generate actionable insights for marketing department.

Database Environment Setup

Initializing the data warehouse for retail analytics.

```
CREATE DATABASE RetailAnalytics;
USE RetailAnalytics;
```

Data Import & Verification

Ensuring row counts match the source CSV files to confirm data integrity.

Step 1:
In MySQL Workbench, you don't need code to load the CSVs:
Step 2:
Right-click your table (Customer_Master) in the Navigator.
Step 3:
Select Table Data Import Wizard.
Step 4:
Choose your CSV file.

Important: Ensure your date formats in the CSV match MySQL's YYYY-MM-DD. If they don't, import them as VARCHAR first and then use STR_TO_DATE() to convert them.

```
-- Checking the data imported correctly
SELECT count(*) FROM customer_master;
SELECT count(*) FROM transactions;
```

PHASE 2

Standardizing Date Formats

Converting string dates (DD-MM-YYYY) into standard SQL **DATE** format for calculations.

```
SET SQL_SAFE_UPDATES=0;

-- Format Transactions
UPDATE Transactions
SET TransactionDate = STR_TO_DATE(TransactionDate, '%d-%m-%Y');
--for permanent changes
ALTER TABLE Transactions MODIFY COLUMN TransactionDate DATE;

-- Format Customer Master
UPDATE customer_master
SET JoinDate = str_to_date(Joindate, '%d-%m-%Y');
--for permanent changes
ALTER TABLE customer_master MODIFY COLUMN joindate DATE;
```

PHASE 3

The Core RFM Calculation

Recency: *Days since last purchase.*
Frequency: *Total number of purchases.*
Monetary: *Total spend in Rupees (₹).*

#In Python, you would use groupby.
#In SQL, we use a Common Table Expression (CTE) to calculate the base metrics and then assign scores using the NTILE() function.

STEP 1 : *Define Metrics and Scores.*

This single query calculates the Recency,Frequency, and Monetary values and assigns a score from 1 to 5 for each.
In this phase,The query looks at the Transactions table and performs three specific math operations for every CustomerID:

Recency Calculation:
It finds the MAX(TransactionDate) (the most recent day they shopped) and subtracts it from a "reference date" to see how many days they have been away.

Frequency Calculation:
It uses COUNT(TransactionDate) to total up how many separate times that customer has made a purchase.

Monetary Calculation:
It uses SUM(TransactionAmount) to add up the total rupees (₹) the customer has spent

```sql
WITH RFM_Base AS (
    SELECT
    CustomerID,
        #Recency:Days since last purchase relative to 'today' (max date + 1).
    DATEDIFF((SELECT MAX(TransactionDate) + INTERVAL 1 DAY FROM Transactions), MAX(TransactionDate))AS Recency,
        #Frequency:Total count of transactions
    COUNT(TransactionDate) AS Frequency,
        #Monetary:Total spend
    ROUND(SUM(TransactionAmount),2) AS Monetary
    FROM Transactions
    GROUP BY CustomerID
),
RFM_Scores AS (
    SELECT *,
          #NTILE(5)splits the data into 5 equal buckets (quintiles)
          #For Recency a LOWER value is better, so we ORDER BY Recency DESC
      NTILE(5) OVER (ORDER BY Recency DESC) AS R_Score,  -- Lower days = higher score
      NTILE(5) OVER (ORDER BY Frequency ASC) AS F_Score, -- Higher count = higher score
      NTILE(5) OVER (ORDER BY Monetary ASC) AS M_Score   -- Higher spend = higher score
   FROM RFM_Base
)
SELECT * FROM RFM_Scores;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CustomerID	Recency	Frequency	Monetary	R_Score	F_Score	M_Score
CUST10729	36	6	5052.69	3	1	1
CUST10278	18	10	9333.33	4	1	1
CUST10973	12	15	10375.62	4	1	1
CUST10751	137	15	10536.13	1	1	1
CUST10888	42	15	10537.46	3	1	1
CUST10198	17	15	10974.66	4	1	1
CUST10168	50	13	11735.21	2	1	1
CUST10343	41	14	11823.91	3	1	1
CUST10645	10	13	12004.05	5	1	1
CUST10531	96	17	12053.75	1	1	1
CUST10740	41	15	12293.19	3	1	1
CUST10484	274	14	12344.25	1	1	1
CUST10992	60	19	12437.01	2	1	1

Result 1 | Read Only

Scoring Segmentation and Labeling (The Grading System)

We concatenate the scores and apply the business logic rules found in instructions to label the customers.

1. *The "Grading Tool" (NTILE) & In Python, WE use pd.qcut().*
In SQL, we use NTILE(5).
Imagine WE have 1,000 customers.
NTILE(5) does three things automatically :
- a). It sorts all 1,000 customers from best to worst.
- b). It splits them into 5 equal groups (200 people in each group).
- c). It assigns a number: The top group gets a 5, and the bottom group gets a 1.

2. *The Scoring Logic (R, F, and M)*
-- We apply that grading tool to your three metrics, -- but the "sorting" direction changes depending on the metric:

R_Score (Recency):

Logic: A smaller number of days is better because it means they shopped recently.
SQL Command: ORDER BY Recency DESC.
Result: Someone who shopped 1 day ago gets a 5 (Recent),while someone who shopped 100 days ago gets a 1 (Stale).

F_Score (Frequency):

Logic: A higher number of visits is better.
SQL Command: ORDER BY Frequency ASC.
Result: A customer with 50 visits gets a 5 (Frequent),while someone with 1 visit gets a 1 (Rare).

M_Score (Monetary):

Logic: A higher total spend is better.
SQL Command: ORDER BY Monetary ASC.
Result: A big spender gets a 5 (High), and a low spender gets a 1 (Low).

3. Creating the "RFM Cell" (CONCAT)

Once we have the three individual grades (like R=5, F=2, M=1),the query "glues" them together into a single string.
The Query Command: CONCAT(R_Score, F_Score, M_Score).
The Result: It turns the scores into a 3-digit code like "521".

Why?:

It is much easier for a computer (and US!) to identify a "555" as a Champion than to look at three separate columns.

Why is this Phase Important?

Without Phase 4 we just have a list of dates and prices.
After that , every customer has a profile.
"555" means: "They bought yesterday, they buy every week, and they spend a lot."
"111" means: "They bought once a year ago for a very small amount.
1. Sorts all customers based on the metric.
2. Splits them into 5 equal groups (e.g., 20% each).
3. Assigns a grade: 5 = Best, 1 = Worst.

Sorting Directions:

- ***Recency:*** *Lower days = 5 Score (ORDER BY Recency DESC)*
- ***Frequency:*** *Higher visits = 5 Score (ORDER BY Frequency ASC)*
- ***Monetary:*** *Higher spend = 5 Score (ORDER BY Monetary ASC)*

PHASE 4

```sql
SELECT
    CustomerID,
    Recency, Frequency, Monetary,
    CONCAT(R_Score, F_Score, M_Score) AS RFM_SCORE,
    CASE
        WHEN R_Score >= 4 AND F_Score >= 4 AND M_Score >= 4 THEN 'Champions'
        WHEN F_Score >= 4 AND R_Score >= 2 THEN 'Loyal Customers'
        WHEN R_Score >= 4 AND F_Score BETWEEN 2 AND 3 THEN 'Potential Loyalists'
        WHEN R_Score <= 2 AND F_Score >= 3 THEN 'At Risk'
        WHEN R_Score = 1 AND F_Score <= 2 THEN 'Lost'
        WHEN M_Score >= 4 AND F_Score BETWEEN 2 AND 3 THEN 'Big Spenders'
        ELSE 'Others'
    END AS Segment
FROM (
    -- Nest the RFM_Scores logic here
    SELECT *,
        NTILE(5) OVER (ORDER BY Recency DESC) AS R_Score,
        NTILE(5) OVER (ORDER BY Frequency ASC) AS F_Score,
        NTILE(5) OVER (ORDER BY Monetary ASC) AS M_Score
    FROM (
        SELECT CustomerID,
               DATEDIFF((SELECT MAX(TransactionDate) + INTERVAL 1 DAY FROM Transactions), MAX(TransactionDate)) AS Recency,
               COUNT(*) AS Frequency,
               ROUND(SUM(TransactionAmount),2) AS Monetary
        FROM Transactions
        GROUP BY CustomerID
    ) AS base
) AS scored_data;
```

PHASE 4 - OUTPUT

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

CustomerID	Recency	Frequency	Monetary	RFM_SCORE	Segment
CUST10729	36	6	5052.69	311	Others
CUST10278	18	10	9333.33	411	Others
CUST10973	12	15	10375.62	411	Others
CUST10751	137	15	10536.13	111	Lost
CUST10888	42	15	10537.46	311	Others
CUST10198	17	15	10974.66	411	Others
CUST10168	50	13	11735.21	211	Others
CUST10343	41	14	11823.91	311	Others
CUST10645	10	13	12004.05	511	Others
CUST10531	96	17	12053.75	111	Lost
CUST10740	41	15	12293.19	311	Others
CUST10484	274	14	12344.25	111	Lost
CUST10992	60	19	12437.01	211	Others

Result 2 ×

Result Grid | Form Editor | Field Types | Read Only

Retail Analytics - Slide 8

PHASE 5

Insight: Customer Count by Segment

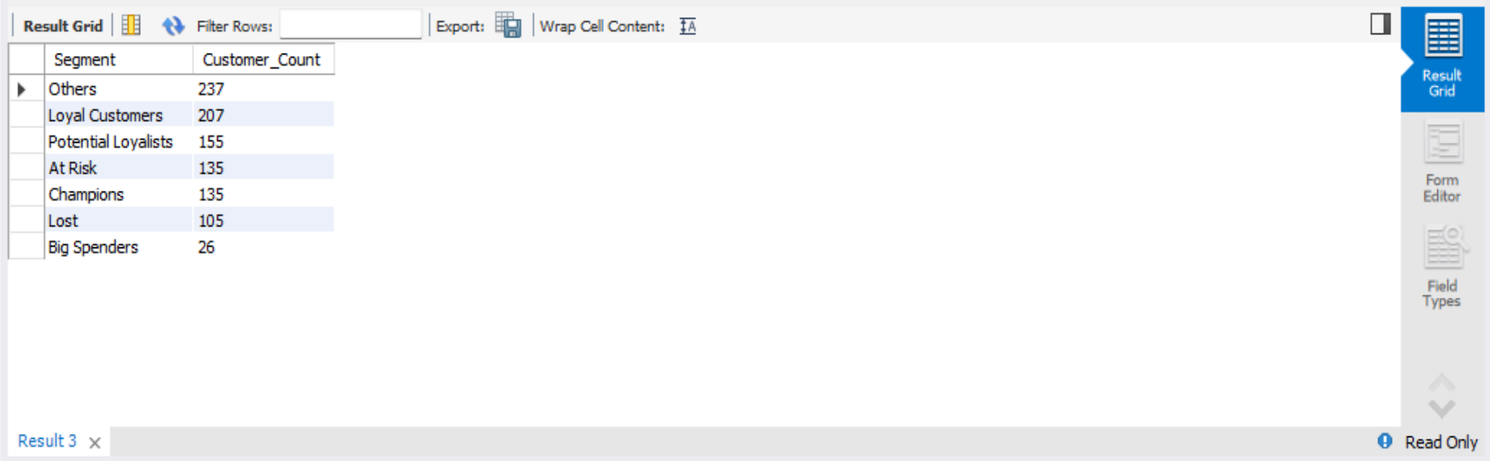
Visualizing the size of each customer group (Bar Chart Data).

> ***Phase 5: Business Insights (Visualizing via SQL)*** *To fulfill the visualization goals of the project,we can run aggregation queries that provide the data for charts.*
>
> ***1. Count of Customers per Segment***
>
> *-TO SIMPLY UNDERSTAND THE QUERIES*
>
> *SELECT Segment, COUNT(*) as Customer_Count*
> *FROM (/* Insert the large query from Phase 4 here */) AS final_segments*
> *GROUP BY Segment*
> *ORDER BY Customer_Count DESC;*
>
> ***Useful for a Bar Chart***

```sql
SELECT Segment, COUNT(*) as Customer_Count
FROM (SELECT
    CustomerID,
    Recency, Frequency, Monetary,
    CONCAT(R_Score, F_Score, M_Score) AS RFM_Cell,
    CASE
        WHEN R_Score >= 4 AND F_Score >= 4 AND M_Score >= 4 THEN 'Champions'
        WHEN F_Score >= 4 AND R_Score >= 2 THEN 'Loyal Customers'
        WHEN R_Score >= 4 AND F_Score BETWEEN 2 AND 3 THEN 'Potential Loyalists'
        WHEN R_Score <= 2 AND F_Score >= 3 THEN 'At Risk'
        WHEN R_Score = 1 AND F_Score <= 2 THEN 'Lost'
        WHEN M_Score >= 4 AND F_Score BETWEEN 2 AND 3 THEN 'Big Spenders'
        ELSE 'Others'
    END AS Segment
            FROM (
                         -- Nest the RFM_Scores logic here

            SELECT *,
               NTILE(5) OVER (ORDER BY Recency DESC) AS R_Score,
               NTILE(5) OVER (ORDER BY Frequency ASC) AS F_Score,
               NTILE(5) OVER (ORDER BY Monetary ASC) AS M_Score
            FROM (
               SELECT CustomerID,
                      DATEDIFF((SELECT MAX(TransactionDate) + INTERVAL 1 DAY FROM Transactions),
                              MAX(TransactionDate)) AS Recency,
                      COUNT(*) AS Frequency,
                              SUM(TransactionAmount) AS Monetary
                      FROM Transactions
                              GROUP BY CustomerID
                ) AS base
                     ) AS scored_data
     ) AS final_segments
GROUP BY Segment
ORDER BY Customer_Count DESC;
```

PHASE 5 - OUTPUT



Segment	Customer_Count
Others	237
Loyal Customers	207
Potential Loyalists	155
At Risk	135
Champions	135
Lost	105
Big Spenders	26

Retail Analytics - Slide 9

Insight: Revenue by Segment

2. Revenue Contribution per Segment

TO SIMPLY UNDERSTAND THE QUERIE IN SHORT

```
SELECT Segment, SUM(Monetary) as Total_Revenue
FROM (/* Insert the large query from Phase 4 here */) AS final_segments
GROUP BY Segment
ORDER BY Total_Revenue DESC;
```

Useful for a Pie Chart or Pareto Analysis

```sql
SELECT Segment, ROUND(SUM(Monetary),2) as Total_Revenue
FROM (SELECT
    CustomerID,
    Recency, Frequency, Monetary,
    CONCAT(R_Score, F_Score, M_Score) AS RFM_SCORE,
    CASE
        WHEN R_Score >= 4 AND F_Score >= 4 AND M_Score >= 4 THEN 'Champions'
        WHEN F_Score >= 4 AND R_Score >= 2 THEN 'Loyal Customers'
        WHEN R_Score >= 4 AND F_Score BETWEEN 2 AND 3 THEN 'Potential Loyalists'
        WHEN R_Score <= 2 AND F_Score >= 3 THEN 'At Risk'
        WHEN R_Score = 1 AND F_Score <= 2 THEN 'Lost'
        WHEN M_Score >= 4 AND F_Score BETWEEN 2 AND 3 THEN 'Big Spenders'
        ELSE 'Others'
    END AS Segment
                FROM (
            -- Nest the RFM_Scores logic here

                    SELECT *,
                NTILE(5) OVER (ORDER BY Recency DESC) AS R_Score,
                NTILE(5) OVER (ORDER BY Frequency ASC) AS F_Score,
                NTILE(5) OVER (ORDER BY Monetary ASC) AS M_Score
                FROM (
                         SELECT CustomerID,
                        DATEDIFF((SELECT MAX(TransactionDate) + INTERVAL 1 DAY FROM Transactions),
                        MAX(TransactionDate)) AS Recency,
                        COUNT(*) AS Frequency,
                        SUM(TransactionAmount) AS Monetary
                        FROM Transactions
                        GROUP BY CustomerID
                        ) AS base
                       ) AS scored_data
     ) AS final_segments
GROUP BY Segment
ORDER BY Total_Revenue DESC;
```

PHASE 5 - OUTPUT

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Segment	Total_Revenue
Loyal Customers	5485063
Others	4224310.04
Champions	3931187.28
At Risk	3391212.66
Potential Loyalists	3315667.19
Lost	2015334.43
Big Spenders	690425.06

Result 4 ×

Result Grid
Form Editor
Field Types

Read Only

Retail Analytics - Slide 10

Conclusion: How this helps the Business

Knowing Your Customers:

We turned 23,000 messy transactions into a clear map showing who our best fans are and who we are about to lose.

Stop Guessing, Start Targeting:

Instead of sending the same generic email to everyone, we can now send specific offers that people actually want to use.

Saving Marketing Costs:

By identifying our top 20% "Big Spenders," we can focus our budget where it actually brings in the most profit.

Growth Mindset:

We now have a reliable system to monitor customer behavior, ensuring our business grows smarter by keeping our best customers happy.

- Visit my LinkedIn Profile
- Visit my GitHub Profile